OP-TEE 실행 흐름 실습 예제

이 문서는 OP-TEE에서 실제 코드가 어떻게 실행되는지 구체적인 예제를 통해 단계별로 설명합니다.

1. Normal World에서 Secure World로의 호출 과정

1.1 전체 호출 흐름

```
Normal World Application

↓ (TEEC_OpenSession)

libteec (Client API)

↓ (ioctl)

Linux Kernel TEE Driver

↓ (SMC call)

ARM TrustZone Monitor

↓ (Secure Monitor Call)

OP-TEE Core (Secure World)

↓ (entry_std.c)

TEE Standard Entry Point

↓ (tee_ta_manager.c)

TA Manager

↓ (user_ta.c)
```

User TA Execution

1.2 SMC 호출 처리 과정

파일: `core/tee/entry_std.c`

```
/*
* Normal World에서 SMC를 통해 호출되는 메인 엔트리 포인트
*/
void  tee_entry_std(struct optee_msg_arg *arg, uint32_t  num_params)
{
TEE_Result res = TEE_SUCCESS;
struct optee_msg_param *params = NULL;


/*
* 1단계: 매개변수 검증
*/
if (num_params && (!arg || !OPTEE_MSG_GET_PARAMS(arg))) {
arg->ret = TEEC_ERROR_BAD_PARAMETERS;
return;
}


params = OPTEE_MSG_GET_PARAMS(arg);
```

```
/*
* 2단계: 명령어별 분기 처리
*/
switch (arg->cmd) {
case OPTEE_MSG_CMD_OPEN_SESSION:
res = entry_open_session(arg, params, num_params);
break;
case OPTEE_MSG_CMD_CLOSE_SESSION:
res = entry_close_session(arg, params, num_params);
break;
case OPTEE_MSG_CMD_INVOKE_COMMAND:
res = entry_invoke_command(arg, params, num_params);
break;
case OPTEE_MSG_CMD_CANCEL:
res = entry_cancel(arg, params, num_params);
break;
case OPTEE_MSG_CMD_REGISTER_SHM:
res = entry_register_shm(arg, params, num_params);
break;
case OPTEE_MSG_CMD_UNREGISTER_SHM:
res = entry_unregister_shm(arg, params, num_params);
break;
default:
```

```
EMSG("Unknown cmd 0x%x", arg->cmd);

res = TEE_ERROR_BAD_PARAMETERS;

break;

}



/*

* 3단계: 결과 반환

*/

arg->ret = res;

arg->ret_origin = TEEC_ORIGIN_TEE;

}
```

1.3 세션 열기 과정 상세 분석

```
static TEE_Result entry_open_session(struct optee_msg_arg *arg,

struct optee_msg_param *params,

uint32_t num_params)

{

TEE_Result res = TEE_SUCCESS;

TEE_ErrorOrigin err_orig = TEE_ORIGIN_TEE;

struct tee_ta_session *s = NULL;

TEE_UUID *uuid = NULL;

TEE_Identity clnt_id = { };
```

```
struct tee_ta_param param = { };



/*

* 1단계: 매개변수 개수 검증

*/

if (num_params != OPTEE_MSG_GET_ARG_SIZE(OPTEE_MSG_OPEN_SESSION_NUM_PARAMS)) {

res = TEE_ERROR_BAD_PARAMETERS;

goto out;

}



/*

* 2단계: UUID 추출

*/

uuid = (TEE_UUID *)&params[0].u.value;

DMSG("Opening session for TA %pUl", (void *)uuid);



/*

* 3단계: 클라이언트 신원 설정

*/

clnt_id.login = params[1].u.value.c;

switch (clnt_id.login) {

case TEE_LOGIN_PUBLIC:

memset(&clnt_id.uuid, 0, sizeof(clnt_id.uuid));
```

```
break;

case TEE_LOGIN_USER:

case TEE_LOGIN_GROUP:

case TEE_LOGIN_APPLICATION:

case TEE_LOGIN_APPLICATION_USER:

case TEE_LOGIN_APPLICATION_GROUP:

clnt_id.uuid = params[1].u.value.b;

break;

default:

res = TEE_ERROR_BAD_PARAMETERS;

goto out;

}



/*

* 4단계: 매개변수 변환

*/

res = copy_in_params(params + 2, num_params - 2, &param);

if (res != TEE_SUCCESS)

goto out;



/*

* 5단계: TA 세션 열기

*/
```

```
res = tee_ta_open_session(&err_orig, &s, &tee_open_sessions,

uuid, &clnt_id, TEE_TIMEOUT_INFINITE,

&param);

if (res != TEE_SUCCESS)

goto out_free_params;



/*

* 6단계: 세션 ID 반환

*/

arg->session = (vaddr_t)s;

arg->ret_origin = err_orig;



/*

* 7단계: 출력 매개변수 복사

*/

res = copy_out_param(params + 2, num_params - 2, &param);



out_free_params:

free_params(&param);

out:

return res;

}
```

2. TA 로딩 및 실행 과정

2.1 User TA 로딩 과정

파일: `core/kernel/user_ta.c`

```
TEE_Result tee_ta_init_user_ta_session(const TEE_UUID *uuid,
struct tee_ta_session *s)
{
TEE_Result res = TEE_SUCCESS;
struct user_ta_ctx *utc = NULL;
struct tee_ta_ctx *ctx = NULL;
struct ts_store_handle *h = NULL;
struct ts_store_ops *ops = NULL;
void *ta_head = NULL;
size_t ta_size = 0;


/*
* 1단계: TA 스토어에서 TA 이미지 찾기
*/
DMSG("Looking for TA %pUl", (void *)uuid);
h = ts_store_open(uuid, &ops);
if (!h) {
DMSG("TA %pUl not found", (void *)uuid);
```

```
return TEE_ERROR_ITEM_NOT_FOUND;

}



/*

* 2단계: TA 헤더 읽기

*/

res = ops->read(h, 0, &ta_head, &ta_size);

if (res != TEE_SUCCESS) {

EMSG("Failed to read TA header");

goto out;

}



/*

* 3단계: TA 서명 검증

*/

res = verify_ta_signature(ta_head, ta_size);

if (res != TEE_SUCCESS) {

EMSG("TA signature verification failed");

goto out;

}



/*

* 4단계: User TA 컨텍스트 할당
```

```
*/

utc = calloc(1, sizeof(struct user_ta_ctx));

if (!utc) {

res = TEE_ERROR_OUT_OF_MEMORY;

goto out;

}



/*

* 5단계: TA 컨텍스트 초기화

*/

ctx = &utc->ta_ctx;

ctx->ts_ctx.uuid = *uuid;

ctx->ts_ctx.ops = &user_ta_ops;

ctx->ref_count = 1;

condvar_init(&ctx->busy_cv);

TAILQ_INIT(&ctx->open_sessions);



/*

* 6단계: TA 이미지 로드

*/

res = load_ta_image(utc, h, ops);

if (res != TEE_SUCCESS) {

EMSG("Failed to load TA image");
```

```
goto out_free_ctx;

}



/*

* 7단계: 메모리 매핑 설정

*/

res = setup_ta_memory_mapping(utc);

if (res != TEE_SUCCESS) {

EMSG("Failed to setup memory mapping");

goto out_free_ctx;

}



/*

* 8단계: 세션과 컨텍스트 연결

*/

s->ts_sess.ctx = &ctx->ts_ctx;

s->ts_sess.handle_scall = s->ts_sess.ctx->ops->handle_scall;



/*

* 9단계: 컨텍스트를 전역 리스트에 추가

*/

TAILQ_INSERT_TAIL(&tee_ctxes, ctx, link);
```

```
goto out;


out_free_ctx:

free(utc);

out:

if (h)

ops->close(h);

return res;

}
```

2.2 TA 메모리 매핑 설정

```
static TEE_Result setup_ta_memory_mapping(struct user_ta_ctx *utc)

{

TEE_Result res = TEE_SUCCESS;

struct ta_head *ta_head = (struct ta_head *)utc->ta_head;

vaddr_t va_start = 0;

size_t va_size = 0;


/*

* 1단계: 가상 주소 공간 크기 계산

*/

va_size = ta_head->stack_size + ta_head->heap_size +
```

```
utc->ta_size + SMALL_PAGE_SIZE; /* Guard page */


/*
* 2단계: 가상 주소 공간 할당
*/
res = vm_alloc(&utc->uctx, &va_start, va_size);
if (res != TEE_SUCCESS) {
EMSG("Failed to allocate VA space");
return res;
}


/*
* 3단계: TA 코드 영역 매핑 (읽기/실행)
*/
res = vm_map(&utc->uctx, &va_start, utc->ta_size,
TEE_MATTR_URX | TEE_MATTR_PR,
utc->ta_mobj, 0);
if (res != TEE_SUCCESS) {
EMSG("Failed to map TA code");
return res;
}
utc->load_addr = va_start;
```

```
/*
* 4단계: 스택 영역 매핑 (읽기/쓰기)
*/
va_start += utc->ta_size + SMALL_PAGE_SIZE; /* Skip guard page */
res = vm_map(&utc->uctx, &va_start, ta_head->stack_size,
TEE_MATTR_URW | TEE_MATTR_PRW,
NULL, 0);
if (res != TEE_SUCCESS) {
EMSG("Failed to map stack");
return res;
}
utc->uctx.stack_ptr = va_start + ta_head->stack_size;


/*
* 5단계: 힙 영역 매핑 (읽기/쓰기)
*/
va_start += ta_head->stack_size;
res = vm_map(&utc->uctx, &va_start, ta_head->heap_size,
TEE_MATTR_URW | TEE_MATTR_PRW,
NULL, 0);
if (res != TEE_SUCCESS) {
EMSG("Failed to map heap");
return res;
```

```
}

return TEE_SUCCESS;
}
```

2.3 TA 엔트리 포인트 호출

```
static TEE_Result user_ta_enter(struct ts_session *session,
enum utee_entry_func func, uint32_t cmd)
{
TEE_Result res = TEE_SUCCESS;
struct utee_params *usr_params = NULL;
uaddr_t usr_stack = 0;
struct user_ta_ctx *utc = to_user_ta_ctx(session->ctx);
struct tee_ta_session *ta_sess = to_ta_session(session);
void *param_va[TEE_NUM_PARAMS] = { NULL };

/*
* 1단계: 재귀 호출 깊이 검사
*/
if (!inc_recursion()) {
res = TEE_ERROR_OUT_OF_MEMORY;
goto out_clr_cancel;
```

```
}

/*
* 2단계: 매개변수 메모리 매핑
*/
if (ta_sess->param) {
res = vm_map_param(&utc->uctx, ta_sess->param, param_va);
if (res != TEE_SUCCESS)
goto out;
}

/*
* 3단계: 현재 세션 설정
*/
ts_push_current_session(session);

/*
* 4단계: 사용자 스택에 매개변수 공간 확보
*/
usr_stack = utc->uctx.stack_ptr;
usr_stack -= ROUNDUP(sizeof(struct utee_params), STACK_ALIGNMENT);
usr_params = (struct utee_params *)usr_stack;
```

```
/*
* 5단계: 매개변수 초기화
*/
if (ta_sess->param)
res = init_utee_param(usr_params, ta_sess->param, param_va);
else
res = clear_user(usr_params, sizeof(*usr_params));


if (res)
goto out_pop_session;


/*
* 6단계: 사용자 모드로 전환하여 TA 실행
*/
res = thread_enter_user_mode(func, kaddr_to_uref(session),
(vaddr_t)usr_params, cmd, usr_stack,
utc->uctx.entry_func, utc->uctx.is_32bit,
&utc->ta_ctx.panicked,
&utc->ta_ctx.panic_code);


/*
* 7단계: VFP 상태 정리
*/
```

```
thread_user_clear_vfp(&utc->uctx);



/*

* 8단계: 패닉 상태 검사

*/

if (utc->ta_ctx.panicked) {

abort_print_current_ts();

DMSG("TA panicked with code 0x%x", utc->ta_ctx.panic_code);

res = TEE_ERROR_TARGET_DEAD;

} else {

/*

* 9단계: 출력 매개변수 업데이트

*/

if (ta_sess->param)

update_from_utee_param(ta_sess->param, usr_params);

}



out_pop_session:

ts_pop_current_session();

/*

* 10단계: 매개변수 메모리 언매핑

*/

if (ta_sess->param)
```

```
vm_unmap_param(&utc->uctx, ta_sess->param);



out:

dec_recursion();

out_clr_cancel:

return res;

}
```

3. 시스템 콜 실행 과정 실습

3.1 암호화 시스템 콜 예제

시나리오: TA에서 AES 암호화를 수행하는 과정

```
/*

* TA 코드에서 TEE_CipherInit() 호출

*/

TEE_Result TEE_CipherInit(TEE_OperationHandle operation,

const void *IV, uint32_t IVLen)

{

/*

* 1단계: 매개변수 준비

*/

struct utee_params params = {
```

```
.types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_VALUE_INPUT,

TEE_PARAM_TYPE_MEMREF_INPUT,

TEE_PARAM_TYPE_NONE,

TEE_PARAM_TYPE_NONE),

.vals = {

(uintptr_t)operation, 0, /* 연산 핸들 */

(uintptr_t)IV, IVLen, /* IV 버퍼 */

0, 0,

0, 0

}

};



/*

* 2단계: 시스템 콜 호출 (어셈블리)

*/

return _utee_cipher_init(&params);

}



/*

* 어셈블리 시스템 콜 래퍼 (libutee)

*/

.global _utee_cipher_init

_utee_cipher_init:
```

```
mov r7, #TEE_SCN_CIPHER_INIT /* 시스템 콜 번호 */

svc #0 /* Supervisor Call */

bx lr /* 반환 */
```

3.2 커널에서의 시스템 콜 처리

```
/*

* 시스템 콜 핸들러에서 cipher_init 처리

*/

TEE_Result syscall_cipher_init(unsigned long state, unsigned long iv,

size_t iv_len)

{

TEE_Result res = TEE_SUCCESS;

struct tee_cryp_state *cs = NULL;

struct ts_session *sess = NULL;

void *iv_buf = NULL;



/*

* 1단계: 현재 세션 획득

*/

sess = ts_get_current_session();

if (!sess)

return TEE_ERROR_BAD_STATE;
```

```
/*
* 2단계: 암호화 상태 검증
*/
res = tee_svc_cryp_get_state(sess, tee_svc_uref_to_vaddr(state), &cs);
if (res != TEE_SUCCESS)
return res;


/*
* 3단계: IV 버퍼 검증 및 복사
*/
if (iv_len > 0) {
/* 사용자 메모리 접근 권한 검사 */
res = tee_mmu_check_access_rights(
to_user_ta_ctx(sess->ctx),
TEE_MEMORY_ACCESS_READ | TEE_MEMORY_ACCESS_ANY_OWNER,
(uaddr_t)iv, iv_len);
if (res != TEE_SUCCESS)
goto out;


/* 커널 버퍼에 복사 */
iv_buf = malloc(iv_len);
if (!iv_buf) {
```

```
res = TEE_ERROR_OUT_OF_MEMORY;

goto out;

}



res = copy_from_user_private(iv_buf, (void *)iv, iv_len);

if (res != TEE_SUCCESS)

goto out;

}



/*

* 4단계: 하드웨어 암호화 엔진 초기화

*/

switch (TEE_ALG_GET_CLASS(cs->algo)) {

case TEE_OPERATION_CIPHER:

res = crypto_cipher_init(cs->ctx, cs->algo, cs->mode,

(uint8_t *)(cs->key1), cs->key1_len,

(uint8_t *)(cs->key2), cs->key2_len,

iv_buf, iv_len);

break;

case TEE_OPERATION_AE:

res = crypto_authenc_init(cs->ctx, cs->algo, cs->mode,

(uint8_t *)(cs->key1), cs->key1_len,

(uint8_t *)(cs->key2), cs->key2_len,
```

```
iv_buf, iv_len, 0, 0, 0);

break;

default:

res = TEE_ERROR_BAD_PARAMETERS;

break;

}



/*

* 5단계: 상태 업데이트

*/

if (res == TEE_SUCCESS)

cs->state = CRYP_STATE_INITIALIZED;



out:

if (iv_buf)

free(iv_buf);

return res;

}
```

4. 메모리 관리 실행 과정

4.1 동적 메모리 할당 과정

```
/*
* TA에서 TEE_Malloc() 호출
*/
void *TEE_Malloc(uint32_t size, uint32_t hint)
{
struct ts_session *sess = ts_get_current_session();
struct user_ta_ctx *utc = to_user_ta_ctx(sess->ctx);
void *ptr = NULL;


/*
* 1단계: 크기 검증
*/
if (!size || size > MAX_MALLOC_SIZE)
return NULL;


/*
* 2단계: 힙에서 메모리 할당
*/
ptr = malloc_from_heap(&utc->heap, size);
if (!ptr) {
/*
* 3단계: 힙 확장 시도
```

```
*/
if (expand_heap(&utc->heap, size)) {
ptr = malloc_from_heap(&utc->heap, size);
}
}

/*
* 4단계: 메모리 초기화 (hint에 따라)
*/
if (ptr && (hint & TEE_MALLOC_FILL_ZERO)) {
memset(ptr, 0, size);
}

return ptr;
}

/*
* 힙에서 메모리 할당
*/
static void *malloc_from_heap(struct malloc_heap *heap, size_t size)
{
void *ptr = NULL;
size_t aligned_size = ROUNDUP(size, MALLOC_ALIGNMENT);
```

```
/*
* 1단계: 뮤텍스 획득
*/
mutex_lock(&heap->mutex);


/*
* 2단계: 적합한 블록 찾기 (First Fit 알고리즘)
*/
struct malloc_block *block = find_free_block(heap, aligned_size);
if (block) {
/*
* 3단계: 블록 분할 (필요한 경우)
*/
if (block->size > aligned_size + sizeof(struct malloc_block)) {
split_block(block, aligned_size);
}


/*
* 4단계: 블록을 사용 중으로 표시
*/
block->used = true;
ptr = (void *)(block + 1);
```

```
/*
* 5단계: 통계 업데이트
*/
heap->allocated += block->size;
if (heap->allocated > heap->max_allocated)
heap->max_allocated = heap->allocated;
}

mutex_unlock(&heap->mutex);
return ptr;
}
```

4.2 페이지 폴트 처리 과정

```
/*
* 페이지 폴트 핸들러
*/
bool abort_handler(uint32_t abort_type, vaddr_t fault_addr,
struct thread_abort_regs *regs)
{
struct ts_session *sess = ts_get_current_session_may_fail();
struct user_ta_ctx *utc = NULL;
```

```
bool handled = false;

if (!sess || !is_user_ta_ctx(sess->ctx))
return false;

utc = to_user_ta_ctx(sess->ctx);

/*
* 1단계: 폴트 주소가 TA 주소 공간 내인지 확인
*/
if (!vm_buf_is_inside_um_private(&utc->uctx, (void *)fault_addr, 1))
return false;

/*
* 2단계: 폴트 타입별 처리
*/
switch (abort_type) {
case ABORT_TYPE_DATA_ABORT:
handled = handle_data_abort(utc, fault_addr, regs);
break;
case ABORT_TYPE_PREFETCH_ABORT:
handled = handle_prefetch_abort(utc, fault_addr, regs);
break;
```

```
default:
EMSG("Unhandled abort type: %u", abort_type);
break;
}

/*
* 3단계: 처리되지 않은 폴트는 TA 종료
*/
if (!handled) {
EMSG("Unhandled page fault at 0x%08x", fault_addr);
utc->ta_ctx.panicked = true;
utc->ta_ctx.panic_code = TEE_ERROR_ACCESS_DENIED;
}

return handled;
}

/*
* 데이터 중단 처리
*/
static bool handle_data_abort(struct user_ta_ctx *utc, vaddr_t fault_addr,
struct thread_abort_regs *regs)
{
```

```
struct vm_region *region = NULL;

TEE_Result res = TEE_SUCCESS;



/*

* 1단계: 폴트 주소가 속한 VM 영역 찾기

*/

region = vm_find_region(&utc->uctx, fault_addr);

if (!region) {

EMSG("No VM region found for address 0x%08x", fault_addr);

return false;

}



/*

* 2단계: 지연 할당 처리

*/

if (region->flags & VM_FLAG_LAZY_ALLOC) {

res = vm_alloc_pages(region, fault_addr);

if (res != TEE_SUCCESS) {

EMSG("Failed to allocate pages: 0x%x", res);

return false;

}

return true;

}
```

```
/*
* 3단계: Copy-on-Write 처리
*/
if (region->flags & VM_FLAG_COW) {
res = vm_handle_cow(region, fault_addr);
if (res != TEE_SUCCESS) {
EMSG("Failed to handle COW: 0x%x", res);
return false;
}
return true;
}


/*
* 4단계: 스택 확장 처리
*/
if (region->flags & VM_FLAG_STACK) {
res = vm_expand_stack(region, fault_addr);
if (res != TEE_SUCCESS) {
EMSG("Failed to expand stack: 0x%x", res);
return false;
}
return true;
```

```
}

return false;
}
```

5. 디버깅 및 추적 예제

5.1 함수 추적 (ftrace) 사용

```
/*
* ftrace 초기화
*/
void ftrace_init(struct ts_session *sess)
{
struct ftrace_buf *fbuf = NULL;
size_t buf_size = CFG_FTRACE_BUF_SIZE;

/*
* 1단계: ftrace 버퍼 할당
*/
fbuf = calloc(1, sizeof(struct ftrace_buf));
if (!fbuf)
return;
```

```
fbuf->buf = malloc(buf_size);
if (!fbuf->buf) {
free(fbuf);
return;
}

/*
* 2단계: 버퍼 초기화
*/
fbuf->buf_size = buf_size;
fbuf->head = 0;
fbuf->enabled = true;
sess->fbuf = fbuf;

DMSG("ftrace initialized for session %p", sess);
}

/*
* 함수 진입 추적
*/
void __cyg_profile_func_enter(void *this_fn, void *call_site)
{
```

```
struct ts_session *sess = ts_get_current_session_may_fail();

struct ftrace_buf *fbuf = NULL;

uint64_t timestamp = 0;



if (!sess || !sess->fbuf || !sess->fbuf->enabled)

return;



fbuf = sess->fbuf;

timestamp = read_cntpct();



/*

* 추적 정보 기록

*/

if (fbuf->head < fbuf->buf_size - sizeof(struct ftrace_entry)) {

struct ftrace_entry *entry =

(struct ftrace_entry *)(fbuf->buf + fbuf->head);

entry->func_addr = (uintptr_t)this_fn;

entry->call_site = (uintptr_t)call_site;

entry->timestamp = timestamp;

entry->type = FTRACE_ENTER;

fbuf->head += sizeof(struct ftrace_entry);

}

}
```

```
/*
* 함수 종료 추적
*/
void __cyg_profile_func_exit(void *this_fn, void *call_site)
{
struct ts_session *sess = ts_get_current_session_may_fail();
struct ftrace_buf *fbuf = NULL;
uint64_t timestamp = 0;

if (!sess || !sess->fbuf || !sess->fbuf->enabled)
return;

fbuf = sess->fbuf;
timestamp = read_cntpct();

/*
* 추적 정보 기록
*/
if (fbuf->head < fbuf->buf_size - sizeof(struct ftrace_entry)) {
struct ftrace_entry *entry =
(struct ftrace_entry *)(fbuf->buf + fbuf->head);
entry->func_addr = (uintptr_t)this_fn;
```

```
entry->call_site = (uintptr_t)call_site;

entry->timestamp = timestamp;

entry->type = FTRACE_EXIT;

fbuf->head += sizeof(struct ftrace_entry);

}

}
```

5.2 성능 측정 예제

```
/*

* 암호화 연산 성능 측정

*/

TEE_Result measure_crypto_performance(void)

{

TEE_Result res = TEE_SUCCESS;

TEE_OperationHandle op = TEE_HANDLE_NULL;

TEE_ObjectHandle key = TEE_HANDLE_NULL;

uint8_t plaintext[1024] = { 0 };

uint8_t ciphertext[1024] = { 0 };

uint32_t ciphertext_len = sizeof(ciphertext);

TEE_Time start_time, end_time;

uint32_t elapsed_ms = 0;
```

```
/*
* 1단계: 시작 시간 기록
*/
TEE_GetSystemTime(&start_time);


/*
* 2단계: AES 키 생성
*/
res = TEE_AllocateTransientObject(TEE_TYPE_AES, 256, &key);
if (res != TEE_SUCCESS)
goto out;


res = TEE_GenerateKey(key, 256, NULL, 0);
if (res != TEE_SUCCESS)
goto out;


/*
* 3단계: 암호화 연산 할당
*/
res = TEE_AllocateOperation(&op, TEE_ALG_AES_ECB_NOPAD,
TEE_MODE_ENCRYPT, 256);
if (res != TEE_SUCCESS)
goto out;
```

```
/*
* 4단계: 키 설정
*/
res = TEE_SetOperationKey(op, key);
if (res != TEE_SUCCESS)
goto out;


/*
* 5단계: 암호화 수행 (1000회 반복)
*/
for (int i = 0; i < 1000; i++) {
ciphertext_len = sizeof(ciphertext);
res = TEE_CipherDoFinal(op, plaintext, sizeof(plaintext),
ciphertext, &ciphertext_len);
if (res != TEE_SUCCESS)
goto out;
}


/*
* 6단계: 종료 시간 기록 및 계산
*/
TEE_GetSystemTime(&end_time);
```

```
elapsed_ms = (end_time.seconds - start_time.seconds) * 1000 +

(end_time.millis - start_time.millis);



IMSG("AES encryption (1000 blocks): %u ms", elapsed_ms);

IMSG("Throughput: %u KB/s",

(1000 * sizeof(plaintext)) / elapsed_ms);



out:

if (op != TEE_HANDLE_NULL)

TEE_FreeOperation(op);

if (key != TEE_HANDLE_NULL)

TEE_FreeTransientObject(key);



return res;

}
```

이 문서는 OP-TEE의 실제 코드 실행 과정을 구체적인 예제를 통해 보여줍니다. 각 단계별로 실제 함수 호출과 데이터 흐름을 따라가면서 시스템의 동작을 이해할 수 있도록 구성했습니다.

6. 완전한 TA 개발 실습 예제

6.1 보안 저장소 TA 구현

파일: `secure_storage_ta.c`

```
#include <tee_internal_api.h>
```

```
#include <tee_internal_api_extensions.h>

#include <secure_storage_ta.h>


/*

* TA 전역 변수

*/

static TEE_ObjectHandle persistent_obj = TEE_HANDLE_NULL;

static uint8_t temp_buffer[MAX_BUFFER_SIZE];


/*

* TA 생성 시 호출되는 함수

*/

TEE_Result TA_CreateEntryPoint(void)

{

DMSG("Secure Storage TA: Create Entry Point");

/*

* 1단계: 전역 리소스 초기화

*/

memset(temp_buffer, 0, sizeof(temp_buffer));

/*

* 2단계: 암호화 알고리즘 지원 확인

*/

if (!TEE_IsAlgorithmSupported(TEE_ALG_AES_GCM, TEE_ECC_CURVE_NONE)) {
```

```
EMSG("AES-GCM not supported");
return TEE_ERROR_NOT_SUPPORTED;
}
IMSG("Secure Storage TA initialized successfully");
return TEE_SUCCESS;
}


/*
* TA 소멸 시 호출되는 함수
*/
void TA_DestroyEntryPoint(void)
{
DMSG("Secure Storage TA: Destroy Entry Point");
/*
* 리소스 정리
*/
if (persistent_obj != TEE_HANDLE_NULL) {
TEE_CloseObject(persistent_obj);
persistent_obj = TEE_HANDLE_NULL;
}
memset(temp_buffer, 0, sizeof(temp_buffer));
}
```

```
/*
* 세션 열기 시 호출되는 함수
*/
TEE_Result TA_OpenSessionEntryPoint(uint32_t param_types,
TEE_Param params[4],
void **sess_ctx)
{
uint32_t exp_param_types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_VALUE_INPUT,
TEE_PARAM_TYPE_NONE,
TEE_PARAM_TYPE_NONE,
TEE_PARAM_TYPE_NONE);
DMSG("Secure Storage TA: Open Session");
/*
* 1단계: 매개변수 타입 검증
*/
if (param_types != exp_param_types) {
EMSG("Bad parameters");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 2단계: 클라이언트 인증 (예제)
*/
uint32_t client_id = params[0].value.a;
```

```
if (client_id == 0) {
EMSG("Invalid client ID");
return TEE_ERROR_ACCESS_DENIED;
}
/*
* 3단계: 세션 컨텍스트 할당
*/
struct session_context *ctx = TEE_Malloc(sizeof(struct session_context), 0);
if (!ctx) {
EMSG("Failed to allocate session context");
return TEE_ERROR_OUT_OF_MEMORY;
}
/*
* 4단계: 세션 컨텍스트 초기화
*/
ctx->client_id = client_id;
ctx->session_key = TEE_HANDLE_NULL;
ctx->authenticated = false;
*sess_ctx = ctx;
IMSG("Session opened for client %u", client_id);
return TEE_SUCCESS;
}
```

```
/*
* 세션 닫기 시 호출되는 함수
*/
void TA_CloseSessionEntryPoint(void *sess_ctx)
{
struct session_context *ctx = (struct session_context *)sess_ctx;
DMSG("Secure Storage TA: Close Session");
if (ctx) {
/*
* 세션 키 정리
*/
if (ctx->session_key != TEE_HANDLE_NULL) {
TEE_FreeTransientObject(ctx->session_key);
}
/*
* 민감한 데이터 지우기
*/
memset(ctx, 0, sizeof(struct session_context));
TEE_Free(ctx);
}
}


/*
```

```
* 명령 실행 함수
*/
TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx, uint32_t cmd_id,
uint32_t param_types, TEE_Param params[4])
{
struct session_context *ctx = (struct session_context *)sess_ctx;
DMSG("Command ID: %u", cmd_id);
/*
* 세션 컨텍스트 검증
*/
if (!ctx) {
EMSG("Invalid session context");
return TEE_ERROR_BAD_STATE;
}
/*
* 명령별 처리
*/
switch (cmd_id) {
case CMD_AUTHENTICATE:
return cmd_authenticate(ctx, param_types, params);
case CMD_STORE_DATA:
return cmd_store_data(ctx, param_types, params);
case CMD_RETRIEVE_DATA:
```

```
return cmd_retrieve_data(ctx, param_types, params);
case CMD_DELETE_DATA:
return cmd_delete_data(ctx, param_types, params);
case CMD_LIST_OBJECTS:
return cmd_list_objects(ctx, param_types, params);
default:
EMSG("Unknown command: %u", cmd_id);
return TEE_ERROR_BAD_PARAMETERS;
}
}


/*
* 인증 명령 구현
*/
static TEE_Result cmd_authenticate(struct session_context *ctx,
uint32_t param_types, TEE_Param params[4])
{
TEE_Result res = TEE_SUCCESS;
uint32_t exp_param_types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
TEE_PARAM_TYPE_VALUE_OUTPUT,
TEE_PARAM_TYPE_NONE,
TEE_PARAM_TYPE_NONE);
/*
```

```
* 1단계: 매개변수 검증
*/
if (param_types != exp_param_types) {
EMSG("Bad parameters for authenticate");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 2단계: 패스워드 검증
*/
uint8_t *password = (uint8_t *)params[0].memref.buffer;
size_t password_len = params[0].memref.size;
if (!password || password_len == 0) {
EMSG("Invalid password");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 3단계: 해시 계산
*/
TEE_OperationHandle hash_op = TEE_HANDLE_NULL;
uint8_t hash[TEE_SHA256_HASH_SIZE];
uint32_t hash_len = sizeof(hash);
res = TEE_AllocateOperation(&hash_op, TEE_ALG_SHA256, TEE_MODE_DIGEST, 0);
if (res != TEE_SUCCESS) {
```

```
EMSG("Failed to allocate hash operation");
goto out;
}
res = TEE_DigestDoFinal(hash_op, password, password_len, hash, &hash_len);
if (res != TEE_SUCCESS) {
EMSG("Failed to compute hash");
goto out;
}
/*
* 4단계: 저장된 해시와 비교 (예제에서는 하드코딩)
*/
uint8_t expected_hash[TEE_SHA256_HASH_SIZE] = {
0x5e, 0x88, 0x48, 0x98, 0xda, 0x28, 0x04, 0x71,
0x51, 0xd0, 0xe5, 0x6f, 0x8d, 0xc6, 0x29, 0x27,
0x73, 0x60, 0x3d, 0x0d, 0x6a, 0xab, 0xbd, 0xd6,
0x2a, 0x11, 0xef, 0x72, 0x1d, 0x15, 0x42, 0xd8
};
if (TEE_MemCompare(hash, expected_hash, hash_len) == 0) {
ctx->authenticated = true;
params[1].value.a = 1; /* 인증 성공 */
IMSG("Authentication successful for client %u", ctx->client_id);
} else {
ctx->authenticated = false;
```

```
params[1].value.a = 0; /* 인증 실패 */
EMSG("Authentication failed for client %u", ctx->client_id);
res = TEE_ERROR_ACCESS_DENIED;
}
out:
if (hash_op != TEE_HANDLE_NULL)
TEE_FreeOperation(hash_op);
return res;
}


/*
* 데이터 저장 명령 구현
*/
static TEE_Result cmd_store_data(struct session_context *ctx,
uint32_t param_types, TEE_Param params[4])
{
TEE_Result res = TEE_SUCCESS;
uint32_t exp_param_types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
TEE_PARAM_TYPE_MEMREF_INPUT,
TEE_PARAM_TYPE_VALUE_INPUT,
TEE_PARAM_TYPE_NONE);
/*
* 1단계: 인증 확인
```

```
*/
if (!ctx->authenticated) {
EMSG("Client not authenticated");
return TEE_ERROR_ACCESS_DENIED;
}
/*
* 2단계: 매개변수 검증
*/
if (param_types != exp_param_types) {
EMSG("Bad parameters for store_data");
return TEE_ERROR_BAD_PARAMETERS;
}
char *object_id = (char *)params[0].memref.buffer;
size_t object_id_len = params[0].memref.size;
uint8_t *data = (uint8_t *)params[1].memref.buffer;
size_t data_len = params[1].memref.size;
uint32_t flags = params[2].value.a;
if (!object_id || !data || data_len == 0) {
EMSG("Invalid parameters");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 3단계: 암호화 키 생성
```

```
*/
TEE_ObjectHandle aes_key = TEE_HANDLE_NULL;
res = TEE_AllocateTransientObject(TEE_TYPE_AES, 256, &aes_key);
if (res != TEE_SUCCESS) {
EMSG("Failed to allocate AES key");
goto out;
}
res = TEE_GenerateKey(aes_key, 256, NULL, 0);
if (res != TEE_SUCCESS) {
EMSG("Failed to generate AES key");
goto out;
}
/*
* 4단계: 데이터 암호화
*/
TEE_OperationHandle cipher_op = TEE_HANDLE_NULL;
res = TEE_AllocateOperation(&cipher_op, TEE_ALG_AES_GCM, TEE_MODE_ENCRYPT, 256);
if (res != TEE_SUCCESS) {
EMSG("Failed to allocate cipher operation");
goto out;
}
res = TEE_SetOperationKey(cipher_op, aes_key);
if (res != TEE_SUCCESS) {
```

```
EMSG("Failed to set cipher key");
goto out;
}
/*
* 5단계: IV 생성
*/
uint8_t iv[12]; /* GCM IV */
TEE_GenerateRandom(iv, sizeof(iv));
res = TEE_CipherInit(cipher_op, iv, sizeof(iv));
if (res != TEE_SUCCESS) {
EMSG("Failed to initialize cipher");
goto out;
}
/*
* 6단계: 암호화 수행
*/
uint8_t *encrypted_data = TEE_Malloc(data_len + 16, 0); /* +16 for tag */
if (!encrypted_data) {
EMSG("Failed to allocate encrypted buffer");
res = TEE_ERROR_OUT_OF_MEMORY;
goto out;
}
uint32_t encrypted_len = data_len + 16;
```

```
res = TEE_CipherDoFinal(cipher_op, data, data_len, encrypted_data, &encrypted_le
if (res != TEE_SUCCESS) {
EMSG("Failed to encrypt data");
goto out_free_encrypted;
}
/*
* 7단계: 영구 객체 생성
*/
TEE_ObjectHandle persistent_obj = TEE_HANDLE_NULL;
res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
object_id, object_id_len,
flags,
aes_key,
encrypted_data, encrypted_len,
&persistent_obj);
if (res != TEE_SUCCESS) {
EMSG("Failed to create persistent object");
goto out_free_encrypted;
}
/*
* 8단계: IV를 객체 속성으로 저장
*/
res = TEE_WriteObjectData(persistent_obj, iv, sizeof(iv));
```

```
if (res != TEE_SUCCESS) {
EMSG("Failed to write IV");
goto out_close_obj;
}
IMSG("Data stored successfully: %s", object_id);
out_close_obj:
TEE_CloseObject(persistent_obj);
out_free_encrypted:
if (encrypted_data)
TEE_Free(encrypted_data);
out:
if (cipher_op != TEE_HANDLE_NULL)
TEE_FreeOperation(cipher_op);
if (aes_key != TEE_HANDLE_NULL)
TEE_FreeTransientObject(aes_key);
return res;
}


/*
* 데이터 검색 명령 구현
*/
static TEE_Result cmd_retrieve_data(struct session_context *ctx,
uint32_t param_types, TEE_Param params[4])
```

```
{
TEE_Result res = TEE_SUCCESS;
uint32_t exp_param_types = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
TEE_PARAM_TYPE_MEMREF_OUTPUT,
TEE_PARAM_TYPE_NONE,
TEE_PARAM_TYPE_NONE);
/*
* 1단계: 인증 확인
*/
if (!ctx->authenticated) {
EMSG("Client not authenticated");
return TEE_ERROR_ACCESS_DENIED;
}
/*
* 2단계: 매개변수 검증
*/
if (param_types != exp_param_types) {
EMSG("Bad parameters for retrieve_data");
return TEE_ERROR_BAD_PARAMETERS;
}
char *object_id = (char *)params[0].memref.buffer;
size_t object_id_len = params[0].memref.size;
uint8_t *output_buffer = (uint8_t *)params[1].memref.buffer;
```

```
size_t output_buffer_len = params[1].memref.size;
if (!object_id || !output_buffer) {
EMSG("Invalid parameters");
return TEE_ERROR_BAD_PARAMETERS;
}
/*
* 3단계: 영구 객체 열기
*/
TEE_ObjectHandle persistent_obj = TEE_HANDLE_NULL;
res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
object_id, object_id_len,
TEE_DATA_FLAG_ACCESS_READ,
&persistent_obj);
if (res != TEE_SUCCESS) {
EMSG("Failed to open persistent object: %s", object_id);
return res;
}
/*
* 4단계: 객체 정보 획득
*/
TEE_ObjectInfo obj_info;
res = TEE_GetObjectInfo1(persistent_obj, &obj_info);
if (res != TEE_SUCCESS) {
```

```
EMSG("Failed to get object info");
goto out;
}
/*
* 5단계: 암호화된 데이터 읽기
*/
uint8_t *encrypted_data = TEE_Malloc(obj_info.dataSize, 0);
if (!encrypted_data) {
EMSG("Failed to allocate encrypted buffer");
res = TEE_ERROR_OUT_OF_MEMORY;
goto out;
}
uint32_t read_bytes = 0;
res = TEE_ReadObjectData(persistent_obj, encrypted_data,
obj_info.dataSize, &read_bytes);
if (res != TEE_SUCCESS) {
EMSG("Failed to read encrypted data");
goto out_free_encrypted;
}
/*
* 6단계: IV 읽기 (처음 12바이트)
*/
uint8_t iv[12];
```

```
if (read_bytes < sizeof(iv)) {
EMSG("Invalid encrypted data format");
res = TEE_ERROR_CORRUPT_OBJECT;
goto out_free_encrypted;
}
TEE_MemMove(iv, encrypted_data, sizeof(iv));
/*
* 7단계: 복호화 준비
*/
TEE_OperationHandle cipher_op = TEE_HANDLE_NULL;
res = TEE_AllocateOperation(&cipher_op, TEE_ALG_AES_GCM, TEE_MODE_DECRYPT, 256);
if (res != TEE_SUCCESS) {
EMSG("Failed to allocate cipher operation");
goto out_free_encrypted;
}
res = TEE_SetOperationKey(cipher_op, persistent_obj);
if (res != TEE_SUCCESS) {
EMSG("Failed to set cipher key");
goto out_free_op;
}
res = TEE_CipherInit(cipher_op, iv, sizeof(iv));
if (res != TEE_SUCCESS) {
EMSG("Failed to initialize cipher");
```

```
goto out_free_op;
}
/*
* 8단계: 복호화 수행
*/
uint32_t decrypted_len = output_buffer_len;
res = TEE_CipherDoFinal(cipher_op,
encrypted_data + sizeof(iv),
read_bytes - sizeof(iv),
output_buffer, &decrypted_len);
if (res != TEE_SUCCESS) {
EMSG("Failed to decrypt data");
goto out_free_op;
}
/*
* 9단계: 실제 복호화된 크기 반환
*/
params[1].memref.size = decrypted_len;
IMSG("Data retrieved successfully: %s (%u bytes)", object_id, decrypted_len);
out_free_op:
TEE_FreeOperation(cipher_op);
out_free_encrypted:
if (encrypted_data)
```

```
TEE_Free(encrypted_data);

out:

TEE_CloseObject(persistent_obj);

return res;

}
```

6.2 클라이언트 애플리케이션 구현

파일: `secure_storage_client.c`

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <tee_client_api.h>

#include <secure_storage_ta.h>


/*

* 전역 변수

*/

static TEEC_Context ctx;

static TEEC_Session sess;

static bool session_opened = false;


/*
```

```
* 컨텍스트 초기화
*/
static TEEC_Result init_context(void)
{
TEEC_Result res = TEEC_SUCCESS;
/*
* 1단계: TEE 컨텍스트 초기화
*/
res = TEEC_InitializeContext(NULL, &ctx);
if (res != TEEC_SUCCESS) {
printf("TEEC_InitializeContext failed: 0x%x\n", res);
return res;
}
printf("TEE context initialized\n");
return res;
}


/*
* 세션 열기
*/
static TEEC_Result open_session(uint32_t client_id)
{
TEEC_Result res = TEEC_SUCCESS;
```

```
TEEC_UUID ta_uuid = SECURE_STORAGE_TA_UUID;

uint32_t err_origin = 0;

TEEC_Operation op;

/*

* 1단계: 연산 구조체 초기화

*/

memset(&op, 0, sizeof(op));

op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,

TEEC_NONE,

TEEC_NONE,

TEEC_NONE);

op.params[0].value.a = client_id;

/*

* 2단계: 세션 열기

*/

res = TEEC_OpenSession(&ctx, &sess, &ta_uuid,

TEEC_LOGIN_PUBLIC, NULL, &op, &err_origin);

if (res != TEEC_SUCCESS) {

printf("TEEC_OpenSession failed: 0x%x (origin: 0x%x)\n",

res, err_origin);

return res;

}

session_opened = true;
```

```
printf("Session opened for client %u\n", client_id);
return res;
}


/*
* 인증 수행
*/
static TEEC_Result authenticate(const char *password)
{
TEEC_Result res = TEEC_SUCCESS;
TEEC_Operation op;
uint32_t err_origin = 0;
/*
* 1단계: 연산 구조체 초기화
*/
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_VALUE_OUTPUT,
TEEC_NONE,
TEEC_NONE);
op.params[0].tmpref.buffer = (void *)password;
op.params[0].tmpref.size = strlen(password);
/*
```

```
* 2단계: 인증 명령 호출
*/
res = TEEC_InvokeCommand(&sess, CMD_AUTHENTICATE, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Authentication failed: 0x%x (origin: 0x%x)\n",
res, err_origin);
return res;
}
/*
* 3단계: 인증 결과 확인
*/
if (op.params[1].value.a == 1) {
printf("Authentication successful\n");
} else {
printf("Authentication failed - wrong password\n");
return TEEC_ERROR_ACCESS_DENIED;
}
return res;
}


/*
* 데이터 저장
*/
```

```
static TEEC_Result store_data(const char *object_id,
const uint8_t *data, size_t data_len)
{
TEEC_Result res = TEEC_SUCCESS;
TEEC_Operation op;
uint32_t err_origin = 0;
/*
* 1단계: 연산 구조체 초기화
*/
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_MEMREF_TEMP_INPUT,
TEEC_VALUE_INPUT,
TEEC_NONE);
op.params[0].tmpref.buffer = (void *)object_id;
op.params[0].tmpref.size = strlen(object_id);
op.params[1].tmpref.buffer = (void *)data;
op.params[1].tmpref.size = data_len;
op.params[2].value.a = TEE_DATA_FLAG_ACCESS_READ |
TEE_DATA_FLAG_ACCESS_WRITE;
/*
* 2단계: 저장 명령 호출
*/
```

```
res = TEEC_InvokeCommand(&sess, CMD_STORE_DATA, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Store data failed: 0x%x (origin: 0x%x)\n",
res, err_origin);
return res;
}
printf("Data stored successfully: %s (%zu bytes)\n", object_id, data_len);
return res;
}


/*
* 데이터 검색
*/
static TEEC_Result retrieve_data(const char *object_id,
uint8_t *buffer, size_t *buffer_len)
{
TEEC_Result res = TEEC_SUCCESS;
TEEC_Operation op;
uint32_t err_origin = 0;
/*
* 1단계: 연산 구조체 초기화
*/
memset(&op, 0, sizeof(op));
```

```
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_MEMREF_TEMP_OUTPUT,
TEEC_NONE,
TEEC_NONE);
op.params[0].tmpref.buffer = (void *)object_id;
op.params[0].tmpref.size = strlen(object_id);
op.params[1].tmpref.buffer = buffer;
op.params[1].tmpref.size = *buffer_len;
/*
* 2단계: 검색 명령 호출
*/
res = TEEC_InvokeCommand(&sess, CMD_RETRIEVE_DATA, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Retrieve data failed: 0x%x (origin: 0x%x)\n",
res, err_origin);
return res;
}
/*
* 3단계: 실제 데이터 크기 반환
*/
*buffer_len = op.params[1].tmpref.size;
printf("Data retrieved successfully: %s (%zu bytes)\n",
object_id, *buffer_len);
```

```
return res;
}

/*
* 정리 함수
*/
static void cleanup(void)
{
if (session_opened) {
TEEC_CloseSession(&sess);
session_opened = false;
}
TEEC_FinalizeContext(&ctx);
printf("Cleanup completed\n");
}

/*
* 메인 함수
*/
int main(int argc, char *argv[])
{
TEEC_Result res = TEEC_SUCCESS;
uint32_t client_id = 12345;
```

```
const char *password = "hello";

const char *test_data = "This is secret data to be stored securely!";

const char *object_id = "test_object_001";

uint8_t retrieved_buffer[1024];

size_t retrieved_len = sizeof(retrieved_buffer);

printf("=== Secure Storage TA Client ===\n");

/*

* 1단계: 컨텍스트 초기화

*/

res = init_context();

if (res != TEEC_SUCCESS)

goto out;

/*

* 2단계: 세션 열기

*/

res = open_session(client_id);

if (res != TEEC_SUCCESS)

goto out;

/*

* 3단계: 인증

*/

res = authenticate(password);

if (res != TEEC_SUCCESS)
```

```
goto out;
/*
* 4단계: 데이터 저장
*/
res = store_data(object_id, (const uint8_t *)test_data, strlen(test_data));
if (res != TEEC_SUCCESS)
goto out;
/*
* 5단계: 데이터 검색
*/
res = retrieve_data(object_id, retrieved_buffer, &retrieved_len);
if (res != TEEC_SUCCESS)
goto out;
/*
* 6단계: 데이터 검증
*/
retrieved_buffer[retrieved_len] = '\0';
printf("Retrieved data: %s\n", retrieved_buffer);
if (strcmp((char *)retrieved_buffer, test_data) == 0) {
printf("✓ Data integrity verified!\n");
} else {
printf("✗ Data integrity check failed!\n");
res = TEEC_ERROR_CORRUPT_OBJECT;
```

```
}

out:

cleanup();

if (res == TEEC_SUCCESS) {

printf("=== Test completed successfully ===\n");

return 0;

} else {

printf("=== Test failed with error: 0x%x ===\n", res);

return 1;

}

}
```

6.3 빌드 시스템 구성

파일: `Makefile`

```
# TA 빌드 설정

TA_DEV_KIT_DIR ?= $(OPTEE_CLIENT_EXPORT)/include

TEEC_EXPORT ?= $(OPTEE_CLIENT_EXPORT)


# 컴파일러 설정

CROSS_COMPILE ?= arm-linux-gnueabihf-

CC = $(CROSS_COMPILE)gcc

LD = $(CROSS_COMPILE)ld
```

```
OBJCOPY = $(CROSS_COMPILE)objcopy


# 컴파일 플래그
CFLAGS += -Wall -Wextra -Werror
CFLAGS += -I$(TA_DEV_KIT_DIR)
CFLAGS += -I./include
CFLAGS += -fPIC -fno-short-enums
CFLAGS += -fno-common -mno-unaligned-access
CFLAGS += -funwind-tables -DARM32=1


# 링크 플래그
LDFLAGS += -shared -Wl,--no-undefined
LDFLAGS += -Wl,--as-needed -Wl,--gc-sections
LDFLAGS += -L$(TA_DEV_KIT_DIR)/../lib


# 소스 파일
TA_SRCS = secure_storage_ta.c
CLIENT_SRCS = secure_storage_client.c


# 타겟 파일
TA_TARGET = secure_storage.ta
CLIENT_TARGET = secure_storage_client
```

```
# UUID 정의
TA_UUID = 12345678-1234-1234-1234-123456789abc

.PHONY: all clean ta client

all: ta client

# TA 빌드
ta: $(TA_TARGET)

$(TA_TARGET): $(TA_SRCS)
@echo "Building TA: $@"
$(CC) $(CFLAGS) $(LDFLAGS) -o $@ $< -lutils -lutee
@echo "TA built successfully"

# 클라이언트 빌드
client: $(CLIENT_TARGET)

$(CLIENT_TARGET): $(CLIENT_SRCS)
@echo "Building client: $@"
$(CC) $(CFLAGS) -I$(TEEC_EXPORT)/include \
-L$(TEEC_EXPORT)/lib -o $@ $< -lteec
@echo "Client built successfully"
```

```
# 설치
install: all
@echo "Installing TA and client..."
sudo cp $(TA_TARGET) /lib/optee_armtz/$(TA_UUID).ta
sudo cp $(CLIENT_TARGET) /usr/bin/
@echo "Installation completed"

# 테스트 실행
test: install
@echo "Running test..."
$(CLIENT_TARGET)

# 정리
clean:
rm -f $(TA_TARGET) $(CLIENT_TARGET)
rm -f *.o *.so
@echo "Clean completed"

# 디버그 빌드
debug: CFLAGS += -DDEBUG -g -O0
debug: all
```

```
# 릴리스 빌드
release: CFLAGS += -DNDEBUG -O2
release: all
```

6.4 성능 벤치마크 예제

파일: `benchmark.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/time.h>
#include <tee_client_api.h>
#include <secure_storage_ta.h>

/*
* 시간 측정 유틸리티
*/
static double get_time_diff(struct timeval *start, struct timeval *end)
{
return (end->tv_sec - start->tv_sec) +
(end->tv_usec - start->tv_usec) / 1000000.0;
}
```

```
/*
* 암호화 성능 벤치마크
*/
static TEEC_Result benchmark_encryption(TEEC_Session *sess,
size_t data_size, int iterations)
{
TEEC_Result res = TEEC_SUCCESS;
TEEC_Operation op;
uint32_t err_origin = 0;
struct timeval start, end;
double total_time = 0.0;
/*
* 테스트 데이터 생성
*/
uint8_t *test_data = malloc(data_size);
if (!test_data) {
printf("Failed to allocate test data\n");
return TEEC_ERROR_OUT_OF_MEMORY;
}
/*
* 랜덤 데이터 생성
*/
```

```
for (size_t i = 0; i < data_size; i++) {
test_data[i] = rand() & 0xFF;
}
printf("Benchmarking encryption: %zu bytes x %d iterations\n",
data_size, iterations);
/*
* 벤치마크 실행
*/
gettimeofday(&start, NULL);
for (int i = 0; i < iterations; i++) {
char object_id[64];
snprintf(object_id, sizeof(object_id), "bench_obj_%d", i);
/*
* 연산 구조체 초기화
*/
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_MEMREF_TEMP_INPUT,
TEEC_VALUE_INPUT,
TEEC_NONE);
op.params[0].tmpref.buffer = object_id;
op.params[0].tmpref.size = strlen(object_id);
op.params[1].tmpref.buffer = test_data;
```

```
op.params[1].tmpref.size = data_size;
op.params[2].value.a = TEE_DATA_FLAG_ACCESS_READ |
TEE_DATA_FLAG_ACCESS_WRITE;
/*
* 저장 명령 호출
*/
res = TEEC_InvokeCommand(sess, CMD_STORE_DATA, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Store failed at iteration %d: 0x%x\n", i, res);
break;
}
}
gettimeofday(&end, NULL);
total_time = get_time_diff(&start, &end);
/*
* 결과 출력
*/
if (res == TEEC_SUCCESS) {
double throughput = (data_size * iterations) / (total_time * 1024 * 1024);
printf("Encryption benchmark results:\n");
printf(" Total time: %.3f seconds\n", total_time);
printf(" Throughput: %.2f MB/s\n", throughput);
printf(" Average latency: %.3f ms per operation\n",
```

```
(total_time * 1000) / iterations);
}
free(test_data);
return res;
}


/*
* 복호화 성능 벤치마크
*/
static TEEC_Result benchmark_decryption(TEEC_Session *sess,
size_t data_size, int iterations)
{
TEEC_Result res = TEEC_SUCCESS;
TEEC_Operation op;
uint32_t err_origin = 0;
struct timeval start, end;
double total_time = 0.0;
/*
* 출력 버퍼 할당
*/
uint8_t *output_buffer = malloc(data_size);
if (!output_buffer) {
printf("Failed to allocate output buffer\n");
```

```
return TEEC_ERROR_OUT_OF_MEMORY;
}
printf("Benchmarking decryption: %zu bytes x %d iterations\n",
data_size, iterations);
/*
* 벤치마크 실행
*/
gettimeofday(&start, NULL);
for (int i = 0; i < iterations; i++) {
char object_id[64];
snprintf(object_id, sizeof(object_id), "bench_obj_%d", i);
/*
* 연산 구조체 초기화
*/
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_MEMREF_TEMP_OUTPUT,
TEEC_NONE,
TEEC_NONE);
op.params[0].tmpref.buffer = object_id;
op.params[0].tmpref.size = strlen(object_id);
op.params[1].tmpref.buffer = output_buffer;
op.params[1].tmpref.size = data_size;
```

```
/*
* 검색 명령 호출
*/
res = TEEC_InvokeCommand(sess, CMD_RETRIEVE_DATA, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Retrieve failed at iteration %d: 0x%x\n", i, res);
break;
}
}
gettimeofday(&end, NULL);
total_time = get_time_diff(&start, &end);
/*
* 결과 출력
*/
if (res == TEEC_SUCCESS) {
double throughput = (data_size * iterations) / (total_time * 1024 * 1024);
printf("Decryption benchmark results:\n");
printf(" Total time: %.3f seconds\n", total_time);
printf(" Throughput: %.2f MB/s\n", throughput);
printf(" Average latency: %.3f ms per operation\n",
(total_time * 1000) / iterations);
}
free(output_buffer);
```

```
return res;
}

/*
* 메모리 사용량 테스트
*/
static TEEC_Result test_memory_usage(TEEC_Session *sess)
{
TEEC_Result res = TEEC_SUCCESS;
const size_t test_sizes[] = {1024, 4096, 16384, 65536, 262144};
const int num_sizes = sizeof(test_sizes) / sizeof(test_sizes[0]);
printf("=== Memory Usage Test ===\n");
for (int i = 0; i < num_sizes; i++) {
size_t size = test_sizes[i];
printf("Testing with %zu bytes...\n", size);
/*
* 암호화 테스트
*/
res = benchmark_encryption(sess, size, 10);
if (res != TEEC_SUCCESS) {
printf("Encryption test failed for size %zu\n", size);
return res;
}
```

```
/*
* 복호화 테스트
*/
res = benchmark_decryption(sess, size, 10);
if (res != TEEC_SUCCESS) {
printf("Decryption test failed for size %zu\n", size);
return res;
}
printf("\n");
}
return res;
}


/*
* 동시성 테스트
*/
static TEEC_Result test_concurrency(TEEC_Context *ctx)
{
TEEC_Result res = TEEC_SUCCESS;
const int num_sessions = 5;
TEEC_Session sessions[num_sessions];
TEEC_UUID ta_uuid = SECURE_STORAGE_TA_UUID;
uint32_t err_origin = 0;
```

```
printf("=== Concurrency Test ===\n");
printf("Opening %d concurrent sessions...\n", num_sessions);
/*
* 여러 세션 열기
*/
for (int i = 0; i < num_sessions; i++) {
TEEC_Operation op;
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,
TEEC_NONE,
TEEC_NONE,
TEEC_NONE);
op.params[0].value.a = 10000 + i; /* 고유한 클라이언트 ID */
res = TEEC_OpenSession(ctx, &sessions[i], &ta_uuid,
TEEC_LOGIN_PUBLIC, NULL, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Failed to open session %d: 0x%x\n", i, res);
goto cleanup_sessions;
}
printf("Session %d opened\n", i);
}
/*
* 각 세션에서 동시에 작업 수행
```

```
*/
printf("Performing concurrent operations...\n");
for (int i = 0; i < num_sessions; i++) {
/*
* 인증
*/
TEEC_Operation auth_op;
memset(&auth_op, 0, sizeof(auth_op));
auth_op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_VALUE_OUTPUT,
TEEC_NONE,
TEEC_NONE);
const char *password = "hello";
auth_op.params[0].tmpref.buffer = (void *)password;
auth_op.params[0].tmpref.size = strlen(password);
res = TEEC_InvokeCommand(&sessions[i], CMD_AUTHENTICATE,
&auth_op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Authentication failed for session %d: 0x%x\n", i, res);
goto cleanup_sessions;
}
/*
* 데이터 저장
```

```
*/
char test_data[256];
char object_id[64];
snprintf(test_data, sizeof(test_data),
"Concurrent test data from session %d", i);
snprintf(object_id, sizeof(object_id), "concurrent_obj_%d", i);
TEEC_Operation store_op;
memset(&store_op, 0, sizeof(store_op));
store_op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_MEMREF_TEMP_INPUT,
TEEC_VALUE_INPUT,
TEEC_NONE);
store_op.params[0].tmpref.buffer = object_id;
store_op.params[0].tmpref.size = strlen(object_id);
store_op.params[1].tmpref.buffer = test_data;
store_op.params[1].tmpref.size = strlen(test_data);
store_op.params[2].value.a = TEE_DATA_FLAG_ACCESS_READ |
TEE_DATA_FLAG_ACCESS_WRITE;
res = TEEC_InvokeCommand(&sessions[i], CMD_STORE_DATA,
&store_op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Store failed for session %d: 0x%x\n", i, res);
goto cleanup_sessions;
```

```
}
printf("Session %d: data stored successfully\n", i);
}
printf("Concurrency test completed successfully\n");
cleanup_sessions:
/*
* 세션 정리
*/
for (int i = 0; i < num_sessions; i++) {
TEEC_CloseSession(&sessions[i]);
}
return res;
}


/*
* 메인 벤치마크 함수
*/
int main(int argc, char *argv[])
{
TEEC_Result res = TEEC_SUCCESS;
TEEC_Context ctx;
TEEC_Session sess;
TEEC_UUID ta_uuid = SECURE_STORAGE_TA_UUID;
```

```
uint32_t err_origin = 0;
printf("=== OP-TEE Secure Storage Benchmark ===\n\n");
/*
* 컨텍스트 초기화
*/
res = TEEC_InitializeContext(NULL, &ctx);
if (res != TEEC_SUCCESS) {
printf("TEEC_InitializeContext failed: 0x%x\n", res);
return 1;
}
/*
* 세션 열기
*/
TEEC_Operation op;
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,
TEEC_NONE,
TEEC_NONE,
TEEC_NONE);
op.params[0].value.a = 99999; /* 벤치마크 클라이언트 ID */
res = TEEC_OpenSession(&ctx, &sess, &ta_uuid,
TEEC_LOGIN_PUBLIC, NULL, &op, &err_origin);
if (res != TEEC_SUCCESS) {
```

```
printf("TEEC_OpenSession failed: 0x%x\n", res);
goto out;
}
/*
* 인증
*/
memset(&op, 0, sizeof(op));
op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
TEEC_VALUE_OUTPUT,
TEEC_NONE,
TEEC_NONE);
const char *password = "hello";
op.params[0].tmpref.buffer = (void *)password;
op.params[0].tmpref.size = strlen(password);
res = TEEC_InvokeCommand(&sess, CMD_AUTHENTICATE, &op, &err_origin);
if (res != TEEC_SUCCESS) {
printf("Authentication failed: 0x%x\n", res);
goto out_close_session;
}
/*
* 벤치마크 실행
*/
printf("=== Performance Benchmark ===\n");
```

```
    res = benchmark_encryption(&sess, 4096, 100);
    if (res != TEEC_SUCCESS)
    goto out_close_session;
    res = benchmark_decryption(&sess, 4096, 100);
    if (res != TEEC_SUCCESS)
    goto out_close_session;
    printf("\n");
    /*
    * 메모리 사용량 테스트
    */
    res = test_memory_usage(&sess);
    if (res != TEEC_SUCCESS)
    goto out_close_session;
    /*
    * 동시성 테스트
    */
    res = test_concurrency(&ctx);
    if (res != TEEC_SUCCESS)
    goto out_close_session;
    printf("\n=== All benchmarks completed successfully ===\n");
    out_close_session:
    TEEC_CloseSession(&sess);
    out:
```

```
TEEC_FinalizeContext(&ctx);

return (res == TEEC_SUCCESS) ? 0 : 1;

}
```

이제 실습 예제 문서가 크게 확장되었습니다. 완전한 TA 구현부터 클라이언트 애플리케이션, 빌드 시스템, 성능 벤치마크까지 포함하여 실제 개발에 바로 사용할 수 있는 수준의 예제를 제공합니다.

다음으로 디버깅 및 트러블슈팅 가이드를 새로 작성하겠습니다.